

Chapter 9

How Expertise Actually Forms

Estimate how long a stage of your learning will take, from hours and your real weekly schedule · explain why a long timeline is an advantage, not a punishment · combine the three ingredients of expertise: study, projects, and feedback over time

Approved chapter · v1

Contents

Chapter 9. How Expertise Actually Forms

Why this matters

In plain English

9.1 The honest timeline

9.2 Why the long timeline is good news

9.3 The three ingredients of expertise

9.4 Deliberate practice

9.5 Building a portfolio as you learn

9.6 Finding feedback and mentors

9.7 Handling plateaus

9.8 What this means for how you use this book

Common mistakes and how to spot them

In the real world: Farah's week seven

Tools

The project: a 12-week learning system

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 9. How Expertise Actually Forms

Part I — The Map

Chapter at a glance

You will learn to: estimate how long a stage of your learning will take, from hours and your real weekly schedule · explain why a long timeline is an advantage, not a punishment · combine the three ingredients of expertise: study, projects, and feedback over time · design deliberate practice sessions instead of hours of passive study · build a portfolio from the work you do while learning · find feedback, peers, and mentors, and ask for help in a way people say yes to · recognize a plateau in your own practice log and change what you do about it.

Before you start: Chapter 8 (the career tree, and choosing your next door).

Time needed: 2–3 hours, including the exercises. The project runs alongside your learning for 12 weeks.

Tools: a spreadsheet or notebook for a practice log. Nothing to install.

Practice data: Farah Khan's fictional 12-week SQL practice log, and the time estimates printed at the start of Chapters 12 and 13. Every calculation shown is checked.

Why this matters

Chapter 8 told you *what* to learn and in what order. This chapter is about *how long* it takes, and *how* the learning turns into ability. It's the chapter that protects you from two opposite mistakes.

The first mistake is quitting too early. In month four, you'll have worked hard, finished chapters, and still feel slow when you open a real dataset. If you believe the promise that anyone can become a data scientist in twelve weeks, that feeling will look like failure. It isn't. It's the normal middle of learning something difficult.

The second mistake is stopping too soon. In year two, you'll be good at your job, and the work will feel comfortable. Comfort is pleasant, and it's also the point where many people stop improving without noticing.

Both mistakes come from not knowing how expertise forms. Once you know, you can plan realistically, practice in a way that works, collect evidence as you go, and recognize a plateau as a signal to change your practice rather than a reason to give up.

In plain English

Think about learning to cook.

- You can read recipes and watch videos. That's **study**. It teaches you what should happen, but on its own it doesn't get dinner on the table.
- You start cooking real meals, with real ingredients that don't behave like the video. The dal sticks to the pan; the rice is undercooked. That's a **project**, and it's where study turns into skill.
- Your family tells you the dal needs less salt. An aunt watches you and says you're adding the tadka too early. Over months, you cook the same dishes many times and start adjusting without thinking. That's **feedback and time**, and it's what turns skill into judgment.
- Some evenings you cook what you already know. That's comfortable, and you don't get better. Other evenings you try one dish that's slightly beyond you, and pay attention to exactly what went wrong. That's the difference between **naive practice** and **deliberate practice**.
- After a few months, your cooking stops improving for a while, however often you cook. That's a **plateau**. The fix is usually to change what you practice, not to cook more of the same.
- If you photograph your best dishes and write down what you changed, you end up with a record of what you can do. That's a **portfolio**.

Everything in this chapter is a version of learning to cook, applied to data work.

9.1 The honest timeline

Let's be plain about time, because a great deal of advertising is not.

Reaching real expertise in this field takes years, not weeks. Becoming a job-ready analyst (tier 1) is a matter of months of steady work for most people who start from zero. Each tier above adds more study, and the top of the tree isn't mainly about study at all. Architects earn their judgment by designing real systems, watching some of them fail, and being responsible when something broke on a Friday night.

No book, course, or bootcamp changes that arithmetic. A course can make your study efficient and your projects well chosen. It can't give you the years of feedback that turn skill into judgment. Anyone who promises to compress a decade into ten weeks is describing a brochure, not a career.

Estimating your own timeline

"Months" and "years" are vague, and vague plans get abandoned. So estimate with numbers you can check.

Every chapter in this book starts with a *Time needed* estimate. Chapter 12, *Databases & SQL Foundations*, says 19–23 hours of reading and practice. Chapter 13, *SQL for Real Analysis*, says 15–20 hours. Together, that's **34–43 hours** to work through the book's two core SQL chapters properly, with exercises.

Now divide by the hours you can *really* give each week, not the hours you wish you had:

Hours per week	Weeks for Chapters 12 and 13
6	$34 \div 6 = 5.7$ to $43 \div 6 = 7.2$ weeks

Hours per week	Weeks for Chapters 12 and 13
10	$34 \div 10 = 3.4$ to $43 \div 10 = 4.3$ weeks

At 6 hours a week, a realistic amount alongside a full-time job, the SQL chapters alone take about six to seven weeks. Part II has eighteen chapters. Working through all of it at that pace is a matter of many months, and that's before the extra practice that makes SQL fluent rather than familiar.

Watch out: chapter hours are not fluency hours. Finishing Chapters 12 and 13 means you can write the queries they teach. Being quick and confident on a messy real dataset takes more: many more questions, answered on data you didn't design. Plan for the chapter hours, then plan for practice beyond them. Section 9.4 shows how to make those hours count.

Try it. Write down the hours you gave to learning last week, counted truthfully (not the hours you had planned). Multiply by four. That's your realistic monthly budget, and the number to plan with.

9.2 Why the long timeline is good news

Here's the reframe that makes the long timeline an advantage instead of a punishment. It's kept from the first edition of this book, because it's still true.

Because it's slow, it's defensible. A skill anyone can pick up in a weekend, everyone picks up in a weekend, and it earns no premium. The very thing that makes this career demanding is what makes it valuable. The years are the moat around your work.

Because it compounds, early effort pays for decades. SQL you learn in month one is still in use in year ten. Chapter 8's skills matrix showed SQL helping to open every door on the tree. Foundations don't expire; they become what everything else stands on.

Because it's layered, you're employable the whole way up. You don't wait years for the payoff. Finish tier 1 and you can be hired as an analyst. Finish tier 4 and you can be an engineer. Every tier is a real job, not only a checkpoint, so you earn while you climb.

9.3 The three ingredients of expertise

Expertise in data work isn't made from study alone. It's an alloy of three ingredients, and leaving any one out gives a weak result.

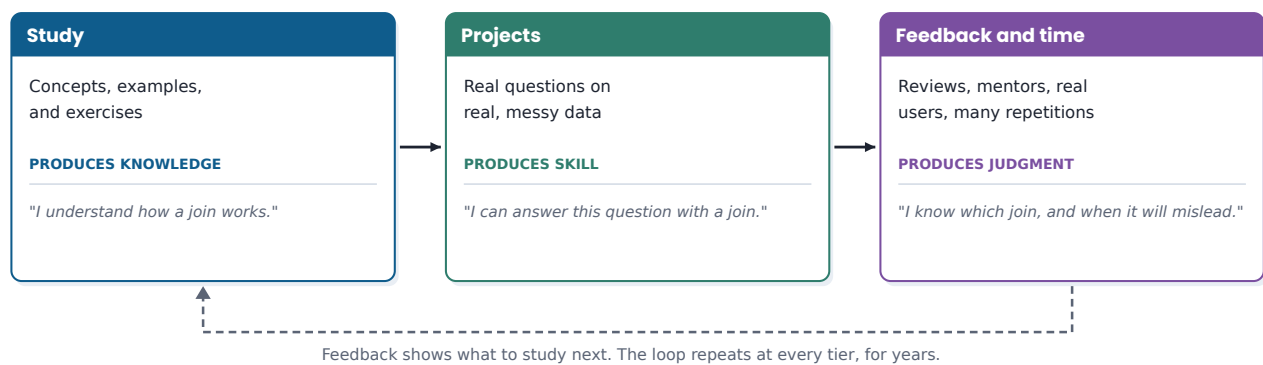


Figure 9.1 – The three ingredients, and what each produces. Most people who stall have plenty of the first and too little of the other two.

1. **Study** gives you **knowledge**: the concepts, from books like this one. It's necessary, but on its own it produces people who can *talk* about data and not *do* it.
2. **Projects** turn knowledge into **skill**: building real things with real, messy data, where nothing matches the example. This is the ingredient most people shortchange, and the one that matters most.
3. **Feedback and time** turn skill into **judgment**: code reviews, mentors, stakeholders who question your numbers, and many repetitions. It can't be rushed.

The three form a loop. Feedback shows you what you don't understand, which tells you what to study next, which you then use on the next project.

People who plateau are usually over-supplied with the first ingredient. They read endlessly and build rarely, because reading feels productive and building feels exposing. Don't be one of them. **Study enough to start building, then build.**

9.4 Deliberate practice

Not all practice is equal. Two people can each spend 100 hours "doing SQL" and end up far apart, because of *how* they spent those hours.

What deliberate practice is

The idea of **deliberate practice** comes from research on expert performers, most associated with the psychologist K. Anders Ericsson, whose studies of musicians in the early 1990s compared how the best and the merely good had practiced. Deliberate practice is practice designed specifically to improve performance, rather than practice that repeats what you can already do.

It has five features.

- **A specific goal.** Not "do some SQL", but "write a query that keeps customers who have never ordered".

- **Difficulty slightly beyond your current ability.** Hard enough that you might fail, not so hard that you can't tell why.
- **Full attention,** for a limited time. Short, focused sessions beat long, distracted ones.
- **Immediate feedback.** You find out quickly whether you were right, and *why* you weren't.
- **Repetition with refinement.** You try again, changed by what the feedback told you, and you come back to it later.

Naive practice is the opposite: doing what's comfortable, without a goal or feedback, and counting the hours.

	Naive practice	Deliberate practice
Goal	"Do some SQL tonight"	"Write a LEFT JOIN that keeps never-ordered customers"
Difficulty	What already feels comfortable	Slightly beyond what you can do unaided
Attention	Half-watching a video, phone nearby	Full focus for a short, fixed block
Feedback	None, or "it ran, so it's right"	Compare with a known answer; find why it differs
Repetition	Move on after one success	Redo it tomorrow without looking; vary it
Record	Nothing written down	A log: what was hard, what to try next

Same hours, different results: deliberate practice spends them where you're weakest, and checks every attempt.

Figure 9.2 – Naive and deliberate practice. The hours can be identical; what you do in them is not.

What the research does and doesn't say

You may have heard that expertise takes "10,000 hours". That number was popularized by Malcolm Gladwell's book *Outliers* (2008), drawing on Ericsson's research. Ericsson himself later objected that the number was an average from one group of musicians, not a threshold, and that the *kind* of practice mattered far more than the count.

Later research also found that practice isn't the whole story. A 2014 meta-analysis by Brooke Macnamara, David Hambrick, and Frederick Oswald, combining many earlier studies, found that the amount of deliberate practice explained 26% of the differences in performance in games, 21% in music, 18% in sports, 4% in education, and less than 1% in professions. The authors concluded that deliberate practice is important, but not as important as had been argued. Researchers still disagree about how to measure practice fairly, so treat the exact percentages with care.

Two honest conclusions follow for you.

- **Practice matters, and its quality matters most.** Nobody in the research became good without a great deal of practice.
- **Practice isn't everything, especially at work.** In professions, what you practice *on* (real problems, real consequences, real feedback) and who you learn *from* matter a great deal. That's why projects and feedback sit beside practice in the three ingredients, and why the rest of this chapter covers them.

Simplification note. The research above studied chess, music, sport, education, and various professions, not data work specifically. Its lessons transfer as sensible guidance, not as measured facts about analysts.

Deliberate practice for data skills

Here are practice methods that fit the five features, and how this book supports them.

Method	What you do	Where the book helps
Predict, then run	Before running a query or formula, write down the result you expect. Then run it and explain any difference.	<i>Predict-the-result</i> prompts in exercises; the "trap" examples in Chapters 12–13
Rebuild without looking	The day after a worked example, rebuild it from the question alone, without the book open.	Every "plain question first" example
Retrieval before rereading	Before rereading a section, write down what you remember. Then check.	<i>Recap</i> and <i>Key terms</i> sections
Spaced repetition	Return to a skill after a few days, then after a week or two, instead of all at once.	<i>Warm-up</i> exercises, and later chapters that reuse earlier skills
Vary the problem	Answer the same question on a different table, or a different question with the same technique.	<i>Core</i> and <i>Stretch</i> exercises; the pattern library in Chapter 13
Hunt for your mistakes	Keep a list of every mistake you make and its cause. Review it weekly.	<i>Common mistakes</i> tables in every chapter
Explain it aloud	Explain a result to someone else, or to a notebook, as if to a manager.	"What to tell Anita" sections

Research on learning has repeatedly found that testing yourself (retrieval) and spreading practice over time (spacing) produce more durable learning than rereading and cramming, even though rereading *feels* more productive. That feeling is the trap: the comfortable method feels like progress precisely because it asks so little of you.

Watch out: AI assistants can turn practice into watching. If an assistant writes the query every time you're stuck, you practice reading answers, not writing them. Use the assistant *after* your own attempt: to compare, to explain a difference, or to suggest a harder variation. Chapter 6 covers learning with AI assistants without letting them think for you.

9.5 Building a portfolio as you learn

A **portfolio** is a small collection of work that shows what you can do. For a fresher or career switcher, it often matters more than a certificate, because it answers the question a hiring manager really has: *can this person do the work?*

The most common mistake is treating the portfolio as something you build *after* learning. Build it *while* you learn. Every chapter project in this book is designed to become a portfolio piece, and Chapter 27 turns the best of them into a finished analyst portfolio.

What makes a good portfolio piece

A strong piece reads like a short piece of real work, not a tour of a tool. It fits on one page, with the details linked.

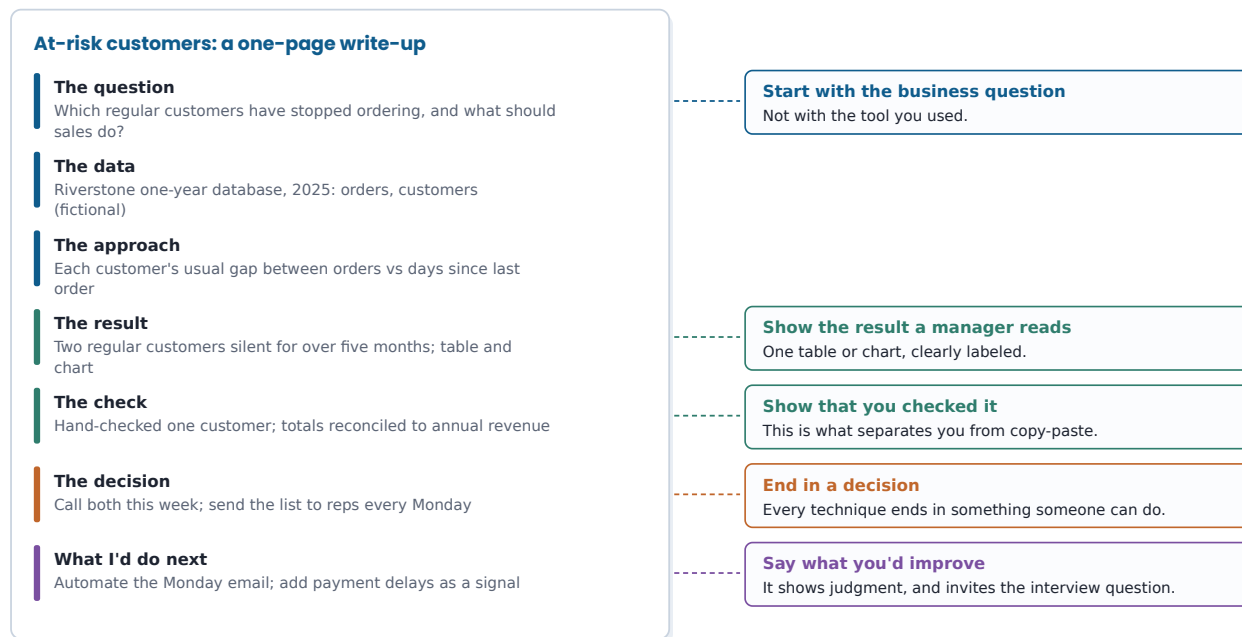


Figure 9.3 — The anatomy of a portfolio piece. The example uses Chapter 13's at-risk customer pattern on the Riverstone one-year database.

1. **The question.** One sentence a manager would ask, in business words.
2. **The data.** What you used, where it came from, and its limits.
3. **The approach.** How you answered it, in plain words, with the code linked.
4. **The result.** One clear table or chart.
5. **The check.** How you know it's right: a hand-checked row, a reconciled total.
6. **The decision.** What someone should do because of it.
7. **What you'd do next.** How you'd improve or automate it.

A portfolio that grows with you

Stage	Pieces worth building	From chapters
Foundations	A classified personal dataset; a process map of a real workflow	1, 3, 7
Analyst core	A cleaned dataset with a data-quality note; a SQL analysis answering three business questions; a dashboard; one automated report	12-20, 27
Advanced analytics	A tested dbt project; an A/B test analysis	30, 32

Stage	Pieces worth building	From chapters
Science or engineering	An end-to-end model with honest evaluation, or a monitored pipeline	44, 46–47
Architecture	A design document with decision records	60, 63

Practical rules

- **Quality over count.** Three pieces that each answer a real question beat ten tutorial copies.
- **Use realistic or public data,** never your employer's confidential data. The Riverstone datasets, public government data, and your own records (anonymized) are all fine.
- **Put it where people can open it in one click:** a GitHub repository with a clear README, a shared folder, or a simple web page. Chapter 26 covers Git and GitHub.
- **Keep the write-up short,** and make the first paragraph readable by a manager who never opens the code.
- **Update it as you climb.** Retire early pieces when better ones replace them.

9.6 Finding feedback and mentors

Feedback is the ingredient you can't give yourself entirely. You can check whether a query runs; it takes someone else to tell you that the question was wrong, or that your chart hides the point.

Where feedback comes from

Different sources give different kinds of feedback. Use several.

Source	What it's good for	Limits
Answer keys and known totals	Is this result correct? Every exercise in this book has a worked answer.	Only for problems with a known answer
The data itself	Reconciling to a trusted total, hand-checking a row	Doesn't tell you if the question was right
AI assistants	Explaining an error; reviewing your code after you've tried	Can be confidently wrong; not a substitute for checking
Study partners and peer groups	Swapping solutions, explaining to each other	Peers may share your blind spots
Online communities	Specific, well-asked technical questions	Slow; answers vary in quality
Colleagues and stakeholders at work	Whether your analysis helps real decisions	You have to show work early, before it's polished
Code review	Readability, correctness, habits	Needs a team or a willing reviewer
Mentors	Direction, judgment, career choices, what to learn next	Their time is limited; their experience is one path

Asking for help well

People say yes to questions that respect their time. A good request for help has five parts.

1. **What you're trying to do**, in one sentence.
2. **What you tried**, with the exact code or formula.
3. **What happened**, with the exact error or output.
4. **What you expected**, and why.
5. **A small example** someone can reproduce, with invented or public data.

Compare "My SQL join isn't working, please help" with "I'm trying to list every customer, including those with no orders. I used a LEFT JOIN with `WHERE o.status <> 'Cancelled'`, and customers with no orders disappear. I expected them to stay with blank order columns. Here are two small tables that show it." The second question is often answered in minutes, and writing it frequently reveals the answer before you send it. (Chapter 12, section 12.10, explains this exact trap.)

Finding a mentor

A **mentor** is someone further along the path who gives you occasional guidance. You don't need a famous one. Someone two or three years ahead of you in your target role is often more useful, because they remember the steps.

- **Start with a specific, small request**, not "will you be my mentor?" For example: "Could I have 20 minutes to ask how you moved from reporting into analytics engineering?"
- **Look close to home first**: senior colleagues, former classmates, alumni of your college, people who answered your questions in a community.
- **Do the work between conversations**. Arrive with what you tried since last time, and one specific question. Mentors keep helping people who use the help.
- **Give something back**. Share a useful article, thank them with a result ("your suggestion about the Monday email worked"), and help people behind you. Explaining to others is also excellent practice.
- **Have more than one**. Different people help with different things: one with SQL habits, another with career decisions.

Real-life example: feedback before it's finished. Many new analysts polish a dashboard for two weeks before showing anyone, and discover in the first meeting that it answers the wrong question. Experienced analysts show a rough version after a day: "Is this the question you meant?" Early feedback feels exposing, and it saves weeks.

9.7 Handling plateaus

Almost everyone who learns a difficult skill hits **plateaus**: stretches where effort continues and visible progress stops. The first edition of this book saved this topic for its closing chapter. It's here now, because you'll meet your first plateau long before the end of the book.

What a plateau looks like in data

Farah Khan, the Riverstone sales executive from Chapter 8, kept a simple log while working through the SQL chapters. Each week she recorded her practice minutes and took a **weekly check**: ten new problems at the same level of difficulty, solved without help, scored out of 10. A fixed check matters, because it measures progress on comparable problems, not on whatever she happened to practice.

Week	1	2	3	4	5	6	7	8	9	10	11	12
Practice minutes	180	200	210	220	240	260	270	240	240	250	240	250
Weekly check (of 10)	3	4	5	6	6	6	6	6	7	8	8	9

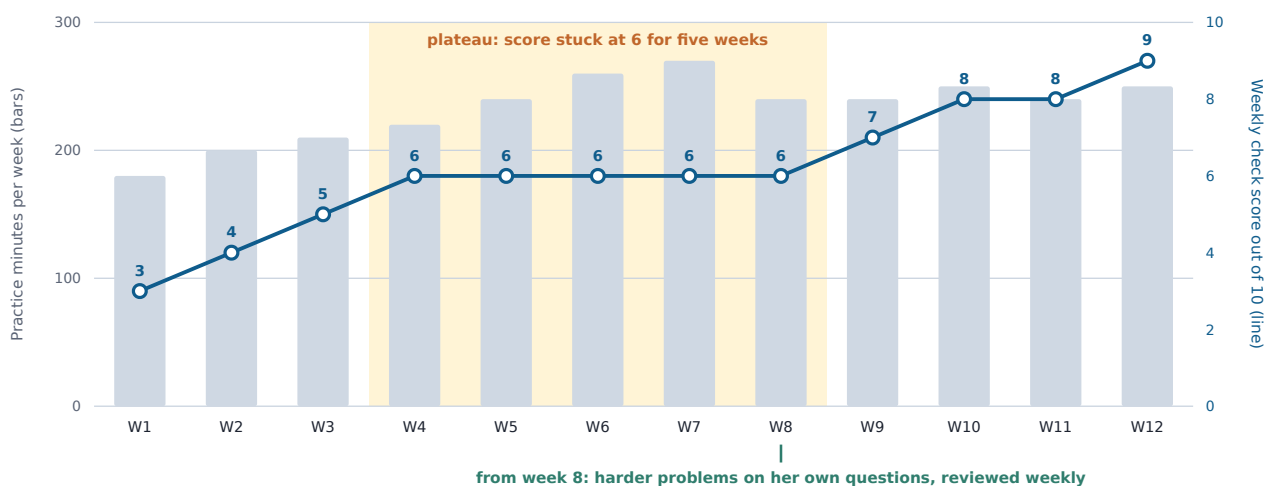


Figure 9.4 – Farah's practice log. During the plateau she practiced more, not less. The score moved again only when she changed how she practiced.

Reading it.

- In weeks 1–4, Farah practiced 810 minutes in total, and her score rose from 3 to 6.
- In weeks 4–8, she practiced 1,230 minutes, and her score didn't move. She responded the way most people do: she practiced *more*. Her weekly minutes rose from 220 in week 4 to 270 in week 7, an increase of 22.7%, with no gain.
- In week 8, she changed *what* she practiced (the story in "In the real world" explains how). In weeks 9–12, she practiced 980 minutes, fewer than in the plateau weeks, and her score rose from 6 to 9.

Over the 12 weeks, she practiced 2,800 minutes, about 46.7 hours: slightly more than the 34–43 hours the book estimates for Chapters 12 and 13. That's consistent with section 9.1: the chapter hours get you through the material, and fluency takes a little more.

Simplification note. Farah's log is fictional and deliberately tidy, to make the pattern clear. Real logs are noisier: a bad week, a holiday, a harder check. Look for a flat stretch over several weeks, not a single low score.

Why plateaus happen

- **The practice became comfortable.** Early on, everything is new, so any practice is deliberate. Later, you drift toward problems you can already solve, because solving them feels good.
- **The difficulty is wrong.** Problems that are too simple teach nothing; problems far too hard give no usable feedback.
- **You're missing one underlying idea.** A single misunderstanding (how NULLs behave, what a row represents) can block a whole family of problems.
- **Tool-hopping.** Switching to a new course or tool every few weeks resets you to the beginner stage, which feels like progress but isn't.
- **Life.** Work pressure, illness, family, and fatigue are real. Some flat weeks are rest, not failure.

What to do about a plateau

1. **Measure before you worry.** Without a fixed check, you can't tell a plateau from a feeling. Start logging.
2. **Change the practice, not only the amount.** More of the same rarely breaks a plateau. Raise the difficulty, switch to your own real questions, or add a new kind of feedback.
3. **Find the missing idea.** Review your mistakes list: do several errors share one cause? Go back to that section and rebuild its examples without looking.
4. **Get outside feedback.** Ask a peer or mentor to watch you solve one problem aloud. They'll often spot a habit you can't see.
5. **Teach it.** Explaining a topic to someone else exposes the parts you only half understand.
6. **Rest deliberately.** A planned week off, followed by a return to the same check, is better than grinding tired.
7. **Don't switch doors because of a plateau alone.** Choose a different role because you'd enjoy its work more (Chapter 8), not because the current skill has gone flat for a month. Every door has its plateaus.

9.8 What this means for how you use this book

Every remaining chapter gives you study *and* a project, on purpose, because the two belong together. Treat the projects as the real curriculum and the prose as the briefing before each one.

In practice, that means:

- **Do the exercises before reading the answers**, and predict results before running code.
- **Keep a practice log** from Part II onward: minutes, a weekly check, and your mistakes list.
- **Turn at least one project per part into a portfolio piece** with the seven-part write-up.
- **Show your work early** to at least one other person each month.
- **Expect plateaus**, and treat them as instructions to change your practice.

Be patient with the third ingredient, feedback and time. It's the one this book can't hand you. Only the work, and the years, can.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Believing a timeline advertised by a course	Feeling like a failure in month three or four	Estimate from chapter hours and your real weekly hours (section 9.1)
Planning with the hours you wish you had	Plans that collapse in the second week	Count last week's real hours; plan with those
Studying without building	Can explain a JOIN, can't answer a question with one	Start a small project as soon as you've learned enough to begin
Rereading instead of testing yourself	Material feels familiar but won't come back without the book	Write what you remember first, then check (retrieval practice)
Counting hours instead of designing practice	Many hours, little change	Use the five features of deliberate practice (section 9.4)
Letting an AI assistant solve every hard step	Can read solutions, can't write them	Attempt first; use the assistant to compare and explain
Building the portfolio "later"	Nothing to show when a good posting appears	Turn chapter projects into one-page pieces as you go
Portfolio of tutorial copies	Every piece looks like everyone else's	Start each piece with a real business question and end with a decision
Asking vague questions for help	Slow or no answers	Use the five-part request: goal, attempt, result, expectation, small example
Waiting until work is polished to show it	Weeks spent on the wrong question	Show a rough version early and ask "is this what you meant?"
Asking a stranger "will you be my mentor?"	Polite silence	Ask a specific, small question; do the work between conversations
Practicing more to break a plateau	More minutes, same results (Figure 9.4)	Change the difficulty, the problems, or the feedback
Switching courses or tools at every plateau	Always a beginner at something	Stay with the skill; change the practice

In the real world: Farah's week seven

By the seventh week of her SQL practice, Farah had stopped enjoying it.

She'd started well. The Riverstone practice databases made sense to her, because she knew the customers behind the rows. Her weekly check had climbed from 3 to 6 in four weeks. Then it stuck. Week 5: 6. Week 6: 6. In week 7 she practiced 270 minutes, the most yet, including two late nights after a long sales day, and scored 6 again.

On Friday afternoon, while they worked on the at-risk list, she told Meera she was thinking of switching to a Power BI course instead. "Maybe SQL isn't for me. I've plateaued."

"Can I see your log?" Meera asked.

Farah opened the sheet. Meera looked at the minutes column, then at the problems Farah had been practicing. Nearly all of them were from the warm-up and core exercises of Chapter 12, which Farah had now done two or three times each.

"You're not bad at SQL," Meera said. "You've become very good at these exercises. You know the answers. Your check has new problems, and that's where you're stuck."

She asked Farah to solve one of the week-7 check problems aloud. Farah got halfway and stopped: the problem needed customers with no orders, and her join kept dropping them. Meera recognized it immediately; she'd tripped over the same thing when she started. "Every one you missed last week has the same shape, doesn't it?"

Farah checked her mistakes list. Eight of the twelve problems she'd got wrong in weeks 5 to 7 involved customers or products that didn't appear in another table. It wasn't a plateau in SQL. It was one idea, the difference between an inner and a left join and where the filter goes, that she'd been working around instead of learning.

They changed three things.

- **Harder, real problems.** Instead of repeating chapter exercises, Farah wrote five questions a week about her own customers, the kind Anita actually asked, and answered them on the practice data.
- **One missing idea, fixed properly.** She went back to Chapter 12, section 12.10, rebuilt its LEFT JOIN examples without looking, and did its trap examples until she could predict every result.
- **Weekly feedback.** Every Friday, she showed Meera one query and explained it aloud, in the ten minutes before they started on the at-risk list.

She also cut her minutes back. The late nights stopped.

Week 8's check was 6 again, and Farah nearly gave up on the new plan. Week 9 was 7. Week 10 was 8. By week 12 she scored 9, having practiced fewer minutes in weeks 9 to 12 than during the plateau. And one of her "own questions" (*which hospitality customers ordered before last year's wedding season but not this year?*) turned into the first portfolio piece she was proud of, written up on one page with the check and the decision.

What made this work.

- **Farah had a log with a fixed check**, so the plateau was a pattern she could see, not a feeling.
 - **Meera looked at how Farah practiced**, not only how much.
 - **The fix was a change of practice**: harder, real problems; one underlying idea fixed; weekly feedback. Not more hours, and not a new course.
 - **The practice produced evidence.** The problems she practiced on became a portfolio piece.
-

Tools

- **A practice log**, in a spreadsheet or notebook. Columns: date, minutes, what you practiced, weekly check score, mistakes and their causes.
 - **A weekly check**: a fixed set of new problems at a steady difficulty. The exercises in later chapters, used for the first time, work well; don't reuse problems you've already practiced.
 - **A place for your portfolio**: a GitHub account (Chapter 26 shows how to use it), a shared folder, or a simple free web page.
 - **A timer**, for short focused practice blocks.
 - **Companion files**: none for this chapter. Farah's log is listed in `checks/ch09_check.py` and `figures/make_figs09.py` if you'd like to chart your own the same way.
-

The project: a 12-week learning system

Goal: a simple system that makes your learning measurable, turns it into evidence, and brings you feedback. Start it now and run it alongside Part II.

Option A: your own learning. Use the next skill on your door plan from Chapter 8.

Privacy reminder. If any practice uses data from your job, keep it private and anonymize anything you show others. Use practice or public data for portfolio pieces.

Option B: Farah's next 12 weeks. Plan Farah's next skill after SQL, Power BI (Chapter 16), using her door plan and this chapter.

Steps

1. **Set your weekly budget.** Count last week's real learning hours. Multiply by 12 for your 12-week budget, and compare it with the *Time needed* of the chapters you plan to cover. Adjust the plan, not the arithmetic.
2. **Build your log** with the columns in Tools.
3. **Design your weekly check:** ten new problems at a steady level, or a timed task (for example, "build this chart from raw data in 30 minutes").
4. **Plan your practice blocks** using at least three methods from the table in section 9.4.
5. **Choose one portfolio piece** to build during the 12 weeks, and outline its seven parts now.
6. **Name your feedback sources:** one peer or study partner, one community, and one person further along the path you'll ask a specific question.
7. **Run it for 12 weeks.** Every Sunday, fill in the log and take the check.
8. **At week 6 and week 12**, chart minutes and scores in the style of Figure 9.4, and write three sentences: what's working, whether you see a plateau, and what you'll change.

Stretch goals

- Keep a mistakes list and group mistakes by cause at week 6. Which single idea would fix the most?
 - Explain one topic to someone else each month, and note what you couldn't explain.
 - Publish your portfolio piece and ask two people for feedback on the write-up, not the code.
-

You've got it when...

- ☐ You can estimate how long a stage of your learning will take from chapter hours and your real weekly hours.
 - ☐ You can explain why a long timeline is defensible, compounding, and layered.
 - ☐ You can name the three ingredients of expertise and what each one produces.
 - ☐ You can describe the five features of deliberate practice and turn a vague goal into a specific one.
 - ☐ You can say what the research on practice does and doesn't show, including why "10,000 hours" is misleading.
 - ☐ You practice by predicting results and rebuilding examples without looking, not by rereading.
 - ☐ You can outline a portfolio piece in the seven parts, and you've started one.
 - ☐ You can write a request for help that's likely to be answered quickly.
 - ☐ You've identified at least two feedback sources and one possible mentor.
 - ☐ You keep a practice log with a fixed weekly check, and can recognize a plateau in it.
 - ☐ You know what to change when you hit a plateau, and why "practice more" usually isn't it.
-

Recap

- **Expertise takes years, not weeks.** Estimate your own timeline with numbers: chapter hours divided by the hours you can really give each week.
- The long timeline is an advantage: skills that take years are **defensible**, they **compound**, and every tier is a real job, so you're employable as you climb.
- Expertise combines three ingredients: **study** (knowledge), **projects** (skill), and **feedback and time** (judgment). Most people who stall have too much of the first.
- **Deliberate practice** has a specific goal, difficulty slightly beyond your ability, full attention, immediate feedback, and repetition with refinement. **Naive practice** counts hours.
- Research shows practice matters but isn't everything; the popular **10,000-hour** rule misreads it. In professions, real problems and real feedback matter a great deal.
- **Retrieval** and **spacing** beat rereading and cramming, even though rereading feels more productive.
- Build a **portfolio** while you learn: a few one-page pieces with a question, data, approach, result, check, decision, and next step.

- Get **feedback** from several sources, ask for help with a five-part request, and find **mentors** with small, specific asks.
 - A **plateau** is flat results despite steady effort. Measure it with a fixed **weekly check**, then change the practice rather than only adding hours.
-

Practice exercises

Warm-up

1. Name the three ingredients of expertise, and what each one produces.
2. Rewrite each vague practice goal as a specific, deliberate one: (a) "Get better at Excel." (b) "Practice SQL this weekend." (c) "Learn Power BI."
3. Classify each activity as naive or deliberate practice, and say why: (a) rereading Chapter 12 for the third time; (b) writing down the expected output of a query before running it; (c) watching a two-hour tutorial while replying to messages; (d) rebuilding yesterday's worked example from the question alone.

Core

4. Arjun can study 45 minutes a day, five days a week. How many hours will he have after 8 weeks? Is that enough to work through Chapters 12 and 13 at the book's estimate of 34–43 hours? If not, how many hours short of the lower estimate is he?
5. Compare two plans over a year: (a) 20 minutes every day of the year; (b) one 3-hour session every Sunday for 52 weeks. Calculate the hours in each. Which plan has more hours, and which would you recommend for learning SQL? Give a reason from section 9.4.
6. In Farah's log, her weekly check score went from 3 in week 1 to 9 in week 12. By what percentage did her score increase? Then explain why her *minutes* in weeks 5–7 are not evidence that she was working badly.
7. Rewrite this request for help using the five-part structure from section 9.6: "Pivot table not working in Google Sheets, totals are wrong, help!!" Invent reasonable details.
8. Outline a portfolio piece in the seven parts from section 9.5, for this question: "Which products sell best to hospitality customers, and should we change what we promote to them?"

Stretch

9. A learner's weekly check scores over ten weeks are 4, 5, 5, 6, 6, 6, 6, 5, 6, 6, while their practice minutes rose every week. Describe what the log shows, list two likely causes from section 9.7, and propose three specific changes.
10. Design one week of deliberate practice (five 40-minute sessions) for someone who has finished Chapter 13 and struggles with window functions. For each session, give the goal, the method, and the source of feedback.
11. A friend says: "Deliberate practice only explains a small share of performance in professions, so practice doesn't matter for data jobs." Using section 9.4, explain what's wrong with that conclusion,

and what the research does suggest.

Think about it (no calculation needed)

- l2. Why might a plateau be *more* likely after several months of learning than in the first few weeks?
 - l3. A career switcher has 8 hours a week and wants to spend all of them on courses until they "know enough" to start a project. What would you tell them, and why?
 - l4. What can a mentor give you that an AI assistant can't, and what can an AI assistant give you that most mentors can't?
-

Key terms

honest timeline · study · project · feedback · knowledge · skill · judgment · deliberate practice · naive practice · 10,000-hour rule · meta-analysis · retrieval practice · spaced repetition · portfolio · portfolio piece · mentor · plateau · practice log · weekly check · mistakes list

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Part II, The Analyst (Chapters 10–27)**, is where you'll use this chapter first. Start your practice log with Chapter 10.
 - **Chapter 6, Setting Up to Learn**, covers your study setup, a sample 6-month analyst plan, and learning with AI assistants without letting them think for you.
 - **Chapter 12, section 12.10**, teaches the LEFT JOIN trap that stalled Farah.
 - **Chapter 26** covers Git and GitHub, where your portfolio can live; **Chapter 27** turns your projects into a finished analyst portfolio.
 - **Chapter 83, The Long Game**, returns to learning over a whole career, including the plateaus of later years.
 - **Part VIII, Chapter 68, How Data Hiring Works**, shows how portfolios are read in hiring, and **Chapter 81** helps you turn your projects and plateaus into strong behavioral interview answers.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. Study produces knowledge (understanding the concepts). **Projects** produce skill (being able to do the work on real, messy data). **Feedback and time** produce judgment (knowing which approach to use, and when it will mislead).

2. Sample answers; any specific, checkable goal is acceptable. (a) "Build a pivot table of Riverstone revenue by month and category, and reconcile its grand total to the total in the source data." (b) "Write three queries that keep customers with no orders, predicting each result before running it." (c) "Build a one-page Power BI report with revenue by month and a slicer by segment, from the Chapter 16 dataset, in 60 minutes." The common weak answer names a topic ("learn LEFT JOINS") instead of a task with a checkable result.

3. (a) Naive: rereading feels productive but doesn't test recall or give feedback. (b) Deliberate: a specific prediction with immediate feedback when the result differs. (c) Naive: divided attention and no feedback. (d) Deliberate: retrieval (rebuilding from memory) with feedback (compare with the worked example).

4. $45 \times 5 \times 8 = 1,800$ minutes = **30 hours**. That's **not enough** for 34–43 hours: he's **4 hours short** of the lower estimate. He could extend the plan by about one more week, or add a longer weekend session. The point of the exercise is to adjust the plan to the arithmetic, not to hope.

5. (a) $20 \times 365 = 7,300$ minutes $\approx$ **121.7 hours**. (b) $3 \times 52 =$ **156 hours**. Plan (b) has more hours, but plan (a) is usually better for learning SQL, because practice spread over many days (spacing) and frequent short, focused sessions produce more durable learning than one long weekly block, where attention fades in the third hour. The best answer might combine them: short daily practice plus a longer weekly project session. Either recommendation is acceptable with a reason from section 9.4.

6. From 3 to 9 is an increase of 6 points: $(9 \div 3 - 1) \times 100 =$ **200%**. Her rising minutes in weeks 5–7 (240, 260, 270) aren't evidence of bad work: she was putting in *more* effort. The log shows that the *kind* of practice had stopped working (she was repeating familiar exercises and working around one missing idea), not that she wasn't trying. The common wrong reading is that a plateau means low effort.

7. A sample answer: "(Goal) I'm building a pivot table in Google Sheets that totals revenue by month. (Attempt) I selected A1:F200 and added Month as rows and Revenue as values, summarized by SUM. (Result) The March total is ₹48,200, but adding the March rows by hand gives ₹52,700. (Expectation) I expected the two to match. (Example) Here's a copy of the sheet with invented data, 12 rows, that shows the same difference." Writing this often reveals the cause, for example rows outside the selected range or revenue stored as text in some rows.

8. Sample outline. **Question:** which products sell best to hospitality customers, and should promotion change? **Data:** Riverstone one-year database (fictional), orders and products for 2025, hospitality segment only; note it excludes returns and tax. **Approach:** revenue and quantity by product for hospitality customers, compared with all customers; share of hospitality revenue per product. **Result:** one table of products ranked by hospitality revenue, with each product's share compared with its share overall. **Check:** hospitality product revenues add up to total hospitality revenue; one order hand-checked. **Decision:** promote the products where hospitality's share is high but sales are still small, and stop promoting products hospitality customers rarely buy. **Next:** repeat by quarter to see seasonality, and send it to the hospitality sales executive monthly. Any outline with all seven parts, a check, and a decision is acceptable; the analysis itself is built in Part II.

9. The log shows a **plateau**: scores rose from 4 to 6 by week 4, then stayed at 5–6 for six weeks while practice minutes kept rising. Likely causes: practice has become **comfortable** (repeating problems already mastered), or a **missing underlying idea** is blocking a family of problems; tool-hopping or

fatigue are also possible. Three changes: (1) review the mistakes list and look for a shared cause, then rebuild that section's examples without looking; (2) replace repeated exercises with new, harder problems, ideally the learner's own real questions; (3) get outside feedback, for example solving one problem aloud for a peer each week, and reduce minutes if fatigue is part of it.

10. Sample week (any design with specific goals, deliberate methods, and a feedback source is acceptable). **Monday:** goal: explain `PARTITION BY` versus `GROUP BY`; method: write a one-paragraph explanation from memory, then check against section 13.3; feedback: the chapter text. **Tuesday:** goal: predict the output of three window queries before running them; method: predict, then run; feedback: the real output. **Wednesday:** goal: rebuild the running-total example from its question alone; method: rebuild without looking; feedback: compare with the chapter's result. **Thursday:** goal: solve two new window problems on a different table; method: vary the problem; feedback: exercise answers. **Friday:** goal: explain one solution aloud to a peer or record it; method: explain it; feedback: the peer's questions, and a note of anything you couldn't explain.

11. The friend misreads the finding. The meta-analysis measured how much of the *differences between people* was explained by the *amount* of deliberate practice they reported; a small share in professions doesn't mean practice is unimportant. Everyone in those professions had already practiced a great deal, which shrinks the differences practice can explain, and on-the-job learning is hard to measure as "deliberate practice". The research suggests that practice matters and its quality matters most, and that in professional work other things also matter: working on real problems, getting real feedback, and learning from experienced people. That's why the three ingredients include projects and feedback, not practice alone.

12. Because early on, everything is new, so almost any practice is at the right difficulty and gives feedback. After a few months, you can already solve the familiar problems, so it's natural to drift toward practice that feels comfortable, and a single missing idea can block progress on the harder problems you now meet. Fatigue and competing commitments also build up over months.

13. Start a small project now, alongside the courses. Knowledge without projects doesn't turn into skill, and it's hard to know what "enough" is until a real problem shows you what's missing. With 8 hours a week, a split such as 5 hours of study and practice and 3 hours on a small project on their own domain gives them all three ingredients, and the project becomes a portfolio piece. The feeling of "not knowing enough" never fully goes away; it's a reason to build, not to wait.

14. A **mentor** can give judgment from experience: which skills matter in your company or city, which role fits you, how to handle a stakeholder, when your question is the wrong question, introductions to people, and encouragement from someone who has been through the same plateaus. An **AI assistant** can give instant, patient explanations at any hour, many variations of practice problems, and a quick first review of code, without using up anyone's time. The assistant can be confidently wrong and doesn't know your situation; the mentor's time is limited and their experience is one path. Use both, and check both.